Profiling Harness

Code architecture, explained end to end

2096 lines / 6 Python modules / 4 profilers / 111 checks

This document walks the harness from the command line down to the process that actually runs your workload. It follows execution order rather than file order, so each piece arrives when you need it.

Section	What it covers
1 - 2	The shape of the system, and the file tree
3 - 4	Entry point, dispatch, and discovery
5 - 6	The two ideas everything rests on: environment layering and the target contract
7 - 9	The main loop, package init, and the bash harness
10 - 12	Process control, output artifacts, retention
13 - 14	Exit codes and the four profilers
15 - 17	Code-size analysis, testing, and judgement calls

How to read this

Every claim about behaviour in this document was verified by running the code, not by reading it. Where a number appears (a race that hits 1 run in 30, a 106 MB trace, a 10-second kill), it was measured. Section 17 lists the places where the implementation deliberately departs from the specification, and why.

SECTION 1

The shape of the system

A pipeline, not a framework

The harness runs a project's workloads (**packages**) under measurement tools (**profilers**), writes timestamped artifacts, and reports a single exit code. Both packages and profilers are discovered from the filesystem, so adding either means creating a directory with two small files - never editing the core.

There is no plugin registry, no dependency injection, no class hierarchy. Each module owns one noun and hands its result to the next:

```
run_profiling.sh          4-line shim: exec python3 lib/cli.py "$@"
       |
       v
.................  cli.py  ...................
:  parse args -> dispatch -> loop over pairs     :
:.............................................:
       |             |              |
       |             |              +--> retention.py    (--remove-output only)
       |             |
       |             +--> discovery.py   what packages and profilers exist?
       |                        |
       |                        +--> envfile.py   what do their .env files mean?
       |
       +--> runner.run_package_init   if PACKAGE_INIT=1
       |
       +--> runner.py      build the argv, write target.sh, spawn ONE bash
       |          |
       |          +--> bash: common.sh + target.sh + package.sh + profiler.sh
       |                        |
       |                        +--> package_pre_run
       |                        +--> profiler_command   <-- builds the command
|                      +--> "${cmd[@]}"       <-- the workload runs here
       |                        +--> profiler_post
       |                        +--> package_post_run   (EXIT trap)
       |
       +--> report.py      meta.json, summary.json, the 'latest' symlink
```

Data flows one way. `discovery` produces immutable value objects; `runner` consumes them and produces a result record; `report` serialises it. Nothing writes back up the chain. That is why the modules can be read in isolation.

The Python/bash split

Orchestration lives in Python; command composition lives in bash. The dividing line is deliberate and it is the single most important design decision in the tool.

Python does	Bash does
Argument parsing, discovery, environment layering, retention, exit-code aggregation, JSON output, process supervision	Building the actual command line, per-package setup, per-profiler invocation
Painful in bash: nested data, JSON, timeouts, signal handling	Natural in bash: quoting, arrays, sourcing config, calling tools

The consequence for a user: **anyone adding a package writes only shell.** There is no Python plugin API to learn. Python 3 was already a hard requirement because three of the four shipped profilers are Python tools, so

it costs nothing.

The core imports only the standard library - verified, not assumed: argparse, dataclasses, hashlib, json, os, pathlib, platform, re, secrets, shlex, shutil, signal, subprocess, sys, tempfile, threading, time, typing. Zero third-party packages.

SECTION 2

The file tree

What each file owns

```
profiling/
  run_profiling.sh        the entry point (a shim)
  install.sh              discovery-driven dependency installer
  config.env              all paths and global defaults
  README.md               user-facing documentation
  .gitignore              output/ and .state/

  lib/                    the core - the only Python in the tree
    cli.py         701    argument parsing, dispatch, the main loop
    runner.py      613    the target contract, bash harnesses, process control
    discovery.py   244    enumerate packages/profilers, typed access to .env
    report.py      228    meta.json, summary.json, tables and JSON listings
    retention.py   175    --remove-output planning and execution
    envfile.py     172    source .env layers, apply precedence
    common.sh             helpers sourced into every leaf hook

  packages/<name>/        a workload to measure
    .env                  declares it
    package.sh            optional hooks

  profilers/<name>/       a way to measure
    .env                  declares it
    profiler.sh           required profiler_command, optional profiler_post

  output/                 run artifacts        (gitignored)
  .state/                 package init stamps  (gitignored)
```

The discovery rule

A directory under `packages/` or `profilers/` is a valid entry **if and only if** it contains a `.env` file and its name does not start with an underscore. Names must match `[a-z0-9][a-z0-9._-]*`; anything else is rejected with a clear error rather than silently ignored.

The underscore prefix is what keeps `_template/` out of the listings while still letting you `cp -r` it. A directory without a `.env` is ignored entirely, so editor droppings and stray folders cause no trouble.

Why state and output are separate trees

`.state/` holds the answer to "has this package been built?". `output/` holds run artifacts. They are separate directories, and the init log is written to `.state/` rather than into a run directory, for one specific reason: **pruning artifacts must never trigger a rebuild.** If the build stamp lived under `output/`, then `--remove-output` would silently cost you a full rebuild on the next run.

It also means you can mount `.state/` as a Docker volume to carry "already built" across containers, while letting `output/` stay ephemeral.

SECTION 3

Entry point and dispatch

run_profiling.sh -> cli.py main()

The shim

```
#!/usr/bin/env bash
set -euo pipefail
exec python3 "$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)/lib/cli.py" "$@"
```

Four lines, and the `exec` matters. It *replaces* the shell process with Python rather than forking, so the process the user sees is the process doing the work. A SIGTERM sent to `run_profiling.sh` lands directly on the Python interpreter - which, as Section 10 explains, is what makes clean cancellation possible at all.

main()

`cli.py:657`. Four steps, in order:

#	Step	Why here
1	`parser.parse_args()`	argparse exits 2 on a bad flag, which is already the code the spec wants
2	`_install_signal_handlers()`	Before anything can spawn a child, so no window exists where a signal orphans work
3	`load_global_env()`	Sources config.env alone. Even --remove-output needs PROFILING_OUTPUT_DIR
4	dispatch	A flat if-chain, because three of four commands are terminal

Terminal actions

`--list-packages`, `--list-profilers` and `--remove-output` each print and return. They never combine with a run, and combining two of them is a usage error. This is why dispatch is a plain if-chain rather than a command table: there are four commands, three of which are one-liners, and the fourth is the entire rest of the program.

```
if args.list_packages:              return cmd_list_packages(env, args.json)
if args.list_profilers:             return cmd_list_profilers(env, args.json)
if args.remove_output is not None:  return cmd_remove_output(args, env)

if args.keep is not None:   raise UsageError("--keep only with --remove-output")
if args.json:               raise UsageError("--json only with --list-*")
return cmd_run(args, env)
```

The two guards before `cmd_run` catch flags that parse fine but mean nothing in context. Silently ignoring `--keep 5` on a run would be worse than refusing it - the user clearly intended something.

The selector grammar

`--package` and `--profiler` both accept four spellings, handled once in `discovery.resolve_selection`:

```
--profiler time                     one name
--profiler time,py-spy              comma-separated
--profiler time --profiler py-spy   repeated flag  (argparse action="append")
--profiler all                      the literal 'all'
```

Mixing `all` with explicit names is an error, not a silent union - `--profiler all,time` almost certainly means the user misunderstands something.

--profiler is mandatory

Omitting it while `--package` is present exits 2 with a message pointing at `--profiler all`. There is no implicit default sweep. Profiling every workload with every tool is expensive, and a harness that does it because you forgot a flag is a harness that burns CI minutes for no reason.

SECTION 4

Discovery

discovery.py - turning directories into typed objects

`discovery.py` does three things: scan the filesystem, load a `.env` stack into an environment dict, and wrap that dict in an object with typed accessors.

Scanning

```
def _scan(root, label, script_name) -> Dict[str, Entry]:
    for child in sorted(root.iterdir()):
        if not child.is_dir() or child.name.startswith(("_", ".")):
            continue                    # _template and hidden dirs
        if not (child / ".env").is_file():
            continue                    # not an entry at all
        if not NAME_RE.match(child.name):
            raise DiscoveryError(...)   # a real mistake - say so
        found[child.name] = Entry(...)
```

`sorted()` gives deterministic order for free. The distinction between the two `continue` branches and the `raise` is deliberate: a directory with no `.env` is not an entry and is none of our business, but a directory that clearly *is* an entry and has an unusable name is a mistake worth stopping for.

Typed access

`Package` and `Profiler` are dataclasses wrapping the merged environment. Their properties are where defaults and parsing live, so no other module has to remember them:

```
@property
def args(self) -> List[str]:
    return shlex.split(self.env.get("PACKAGE_ARGS", ""))      # not .split()

@property
def timeout(self) -> Optional[float]:
    raw = (self.env.get("PACKAGE_TIMEOUT") or "").strip()
    if not raw: return None
    value = float(raw)                  # raises DiscoveryError on garbage
    return None if value <= 0 else value  # 0 means "no timeout"

def supports(self, kind: str) -> bool:
    return not self.kinds or kind in self.kinds   # empty = no restriction
```

`shlex.split` rather than `.split()` is what makes `PACKAGE_ARGS="--input 'my file.json'"` work. This was tested directly against arguments containing spaces, single and double quotes, globs and dollar signs.

Selection and ordering

`resolve_selection` expands selectors into an ordered, de-duplicated list. Ordering follows one rule, applied independently to packages and profilers:

Given	Order
Explicit names	As typed on the command line
`all`	Alphabetical

So `--profiler viztracer,time` runs viztracer first, while `--profiler all` runs a package's list alphabetically no matter how it is written in `PACKAGE_PROFILERS`. The listing output is sorted the same way, so a CI matrix built

from `--list-packages --json` matches what the harness will actually do.

SECTION 5

Environment layering

envfile.py - the first of two central ideas

Every run builds its environment by sourcing four layers in order. Later layers win:

```
1.  config.env                    global paths and defaults
2.  profilers/<profiler>/.env     that profiler's knobs
3.  packages/<package>/.env       the package, ON TOP of the profiler
4.  the real process environment  whatever the caller exported
```

Layer 3 above layer 2 is the whole point

It is what lets a package tune a profiler *for itself*. `packages/my-tool/.env` can set `LINE_PROFILER_TARGETS`, `PYSPY_NATIVE=1` because it has C extensions, or a shallower `VIZTRACER_MAX_DEPTH` because its call graph is deep - all without touching the profiler. This is also why the package is loaded twice per pair, as Section 7 explains.

Why bash does the sourcing

The `.env` files are **sourced by bash, not parsed by Python**. The implementation runs one bash process and reads back its environment:

```
bash -c 'set -a; . "$1"; . "$2"; ...; env -0' _ <files...>
```

`set -a` auto-exports every assignment; `env -0` emits NUL-separated `NAME=VALUE` pairs so values containing newlines survive intact. The payoff is that variable interpolation, command substitution and `PATH="...:$PATH"` all behave exactly as an author would expect, with no parser to write and no subset of shell syntax to document.

The cost is that a `.env` file is code. A stray `$` gets expanded. That is a real trade, and it is documented rather than papered over.

Naming the file that failed

A non-zero exit from the sourcing subshell is a hard error. Getting the filename into that message is harder than it looks, because a *syntax* error kills the shell outright - an `if ! source` guard never runs. An EXIT trap catches both cases:

```
trap '
  if [ "$?" -ne 0 ]; then
    printf "__PROFILING_ENV_FAIL__%s\n" "$__profiling_current" >&2
  fi
' EXIT
```

Python looks for that marker on stderr and reports the offending path. In practice the user sees the filename twice - once from the harness and once from bash's own diagnostic with a line number - which is more useful than either alone.

The one exception to "the real environment wins"

Strict layer-4 precedence has a flaw. If a package writes `PYTHONPATH="$MY_SRC:$PYTHONPATH"`, the overlay would immediately clobber it back to the inherited value, making the line a no-op. So PATH-like variables are treated as *extended* rather than assigned:

```
PATHLIKE = {PATH, PYTHONPATH, LD_LIBRARY_PATH, LD_PRELOAD,
            MANPATH, PKG_CONFIG_PATH, CPATH}
```

```
for key, value in real_env.items():
    if key in PATHLIKE and merged.get(key) != base.get(key):
        continue                # a layer changed it deliberately - keep that
    merged[key] = value         # everything else: the real environment wins
```

Every other variable follows the documented rule exactly. This exception is called out in the README because it is the one place where the four-layer model is not literally true.

SECTION 6

The target contract

runner.py - the second central idea, and the one that makes profilers pluggable

Profilers come in two incompatible shapes, and getting this right is what the whole design turns on:

Family	Example	How it takes the workload
Prefix wrapper	`time`, `py-spy`	Runs the complete command: `/usr/bin/time -v -- python -m tool`
Interpreter replacement	`viztracer`, `kernprof`	**Is** the interpreter: `viztracer -m tool`, never `viztracer python -m tool`

A single argv cannot serve both. So the runner exposes the workload in two forms, written to a generated file that the harness sources:

```
# <RUN_DIR>/target.sh  -- generated, one shlex.quote per element
TARGET_KIND=python-module
TARGET_PYTHON=/repo/.venv/bin/python
TARGET_ARGV=(/repo/.venv/bin/python -m my_tool.cli --input data/sample.json)
TARGET_PYTHON_ARGV=(-m my_tool.cli --input data/sample.json)
```

Variable	Meaning
`TARGET_ARGV`	The complete plain command - exactly what you would run with no profiling. For prefix wrappers.
`TARGET_PYTHON_ARGV`	The same minus the interpreter. This is Python's own trailing CLI shape, so it drops straight in after an interpreter-replacing tool's flags. **Unset when PACKAGE_KIND=exec.**
`TARGET_PYTHON`	The interpreter path. Unset for exec.
`TARGET_KIND`	The package's PACKAGE_KIND.

Why a sourced file rather than environment variables

Bash arrays cannot be exported. Passing an argv through the environment means encoding it - and every encoding scheme eventually meets an argument containing the delimiter. Writing a file with one `shlex.quote` per element and sourcing it sidesteps the problem completely: bash re-parses its own quoting, so the array on the other side is exactly the list Python built.

This was tested with arguments containing semicolons, `$(...)`, backticks, spaces and globs. They arrive as literal argv elements, and nothing is substituted at exec time.

A second benefit: `target.sh` is left in the run directory as an artifact. To reproduce a profiled run by hand, source it and run `"${TARGET_ARGV[@]}"`.

The exec kind, and failing readably

For `PACKAGE_KIND=exec` there is no interpreter to strip, so the generated file emits `unset TARGET_PYTHON_ARGV` rather than an empty array. An empty array would expand to nothing and the profiler would complain about its own arguments, which is a baffling error to debug. Instead `common.sh` provides a guard:

```
require_python_target() {
    if ! profiling_has_python_target; then
        profiling_die \
            "profiler '$PROFILER_NAME' needs a Python target, but package" \
            "'$PACKAGE_NAME' has PACKAGE_KIND=$TARGET_KIND. Use a prefix-style" \
            "profiler (e.g. 'time'), or drop it from PACKAGE_PROFILERS."
    fi
```

```
}
```

Nothing shipped uses `exec` yet - only `time` declares support for it - but the path is plumbed and tested end to end, so adding `perf` or `callgrind` later needs no core change.

The escape hatch

When a command cannot be expressed as kind + entry + args, a package defines `package_command` in `package.sh` and populates the arrays itself. It overrides the declared entry point completely. The harness reads the final argv back from bash after the hook runs, so `meta.json` and `--dry-run` both reflect the override rather than the declaration.

SECTION 7

The main loop

cli.py:345 - cmd_run, the spine of the program

Two nested loops, packages outer and profilers inner, with an eight-step checklist inside. This is the only function in the codebase worth reading carefully.

```
for package_name in package_names:
    base_package = load_package(CONFIG_ENV, entry)          # config + package
    selected, forced = _select_profilers_for_package(...)   # per-package list

    for profiler_name in selected:
        profiler = load_profiler(CONFIG_ENV, ...)           # config + profiler
        package  = load_package(CONFIG_ENV, entry,
                                profiler.entry.env_file)    # config+prof+package
        ...eight steps...
```

Why the package is loaded twice

This is the one genuinely non-obvious line in the file. The package is loaded once *alone* to read `PACKAGE_PROFILERS` - you cannot know which profilers apply until you have read the package - and then again *on top of each profiler's* `.env`. That second layering is what delivers the precedence rule from Section 5. The environment is genuinely different for every (package, profiler) pair, so it must be rebuilt for every pair.

The eight steps

#	Step	Line	On failure
1	Kind compatible?	417	**Skip.** Informational line, recorded as skipped, never affects the exit code
2	Required binaries present?	430	Record failed, exit code 3, continue to the next pair
3	Init needed? (once per package)	458	Skip that package's runs, exit code 4
4	Build target + allocate run id	472	Usage error, exit code 2 - a malformed package is not a run failure
5	Dry run?	479	Print the resolved command and continue - nothing is created
6	Execute	493	Status recorded as failed or timeout, exit code 1
7	Write meta.json, update 'latest'	503	-
8	Aggregate the exit code	507	`max()` - the highest category wins

Three details carry requirements that look bigger than their code

```
exit_code = max(exit_code, EXIT_RUN_FAILED)     # "report the highest code"
init_done: Dict[str, bool] = {}                 # "init at most once per invocation"
if not profiler.supports(package.kind): continue  # "skips never fail the build"
```

A dict, a `max()` and a `continue`. No state machine, no scheduler. The run-everything-then-aggregate behaviour falls out of never breaking the loop.

How the loop body became its own function

The eight steps above used to live inline in `cmd_run`, making it 174 lines. The defence for that was "the ordering of these steps is fragile and only visible when they are adjacent" - which is true, but does not argue for keeping them *inside the loop*. Extracting them into `_run_pair` keeps all eight adjacent and leaves `cmd_run` as a readable 71-line loop.

The real defect was not length. Six different branches each had to remember to do *two* things - append a record **and** raise the exit code - interleaved with the logic, so a branch could silently do one and forget the other. `_run_pair` returns `(record, exit-category)` from every path, so the two can no longer drift apart.

Profiler selection, resolved per package

The rules, in order, from `_select_profilers_for_package`:

Given	Result
`--profiler <name>`	Exactly those, **forced**. A profiler not in the package's list still runs, warns on stderr, and is marked `"forced": true` in meta.json
`--profiler all`	The package's PACKAGE_PROFILERS
...and that is empty	DEFAULT_PROFILERS from config.env
...and that is empty too	Every discovered profiler

Because this resolves *per package*, `--package all --profiler all` is not a cartesian product. A package listing two profilers gets two runs while its neighbour listing three gets three.

SECTION 8

Package init

A flag, and the hook guards itself

Some packages need a build step before they can be profiled: a virtualenv, a compile, `uv sync --frozen`. The package declares that with one flag and puts the work in a hook:

```
# packages/my-service/.env
PACKAGE_INIT=1

# packages/my-service/package.sh
package_init() {
    [ -x .venv/bin/python ] || python3 -m venv .venv
    uv sync --frozen
}
```

`PACKAGE_INIT=1` means *call the hook once, before this package's runs*. 0 or absent means never. That is the entire mechanism.

Why there is no staleness tracking

The hook runs on every invocation, and making it cheap when there is nothing to do is the hook's job -- the guard line above does it. That is deliberate: only the package knows what "already built" means for it, so any check the harness invented on its behalf would be a guess.

An earlier version guessed. It hashed the package's .env, its package.sh and a declared list of lockfiles into a sha256, stored that in `.state/<pkg>/init.json` beside a status field, and compared on the next run. Two problems, and the second is the one that matters:

	What went wrong
Cost	Of 207 lines, about 95 were the cache and 18 actually ran the hook. The stamp needed a path convention, a reader that tolerated corruption, a status field so a failed build was not cached as done, a timestamp, a duration, and a directory to hold it
Correctness	The stamp described a thing it never looked at. Delete the virtualenv and keep the stamp -- exactly what mounting .state as a Docker volume did -- and the harness skipped init, then every run died at exit 127

A guard inside the hook cannot have the second failure, because it tests the artifact rather than a record of the artifact.

--init is the manual trigger, and ignores the flag

```
./run_profiling.sh --init                    # every discovered package
./run_profiling.sh --init --package my-tool  # just one
```

The flag decides whether a *profiling run* builds the package on its own. Asking for `--init` is already saying you want it now, so it does not also require editing the `.env` -- and then remembering to edit it back. Set `PACKAGE_INIT=0` once and build when you choose to.

A package with no hook is skipped by `--init` rather than failing, so it can be pointed at anything. The reverse -- `PACKAGE_INIT=1` with no hook -- *is* an error, because the .env asked for a build step that does not exist.

`./install.sh` calls it after installing dependencies, so one command leaves the box ready to profile. A failed init skips that package's runs, records them as failed and yields exit code 4; nothing is cached, so the next invocation simply tries again.

SECTION 9

The bash harness

Where Python stops and the workload begins

`runner.execute` writes `target.sh`, then spawns **one** bash process running a fixed script. One process, not several, because the hooks share shell state and because a single process is a single process group to kill.

```
. "$PROFILING_ROOT/lib/common.sh"      # helpers: run_artifact, require_bin, ...
. "$RUN_DIR/target.sh"                 # TARGET_ARGV, TARGET_PYTHON_ARGV, ...
. "$PROFILING_PACKAGE_SH"              # package hooks   (if the file exists)
. "$PROFILING_PROFILER_SH"             # profiler hooks  (if the file exists)

declare -F package_command  && package_command      # escape hatch may override
printf '%s\0' "${TARGET_ARGV[@]}" > "$PROFILING_ARGV_FILE"   # report the truth

declare -F profiler_command || { profiling_error "no profiler_command"; exit 78; }

cmd=(); profiler_command               # build it
printf '%s\0' "${cmd[@]}" > "$COMMAND_FILE"
[ "$PROFILING_RESOLVE_ONLY" = 1 ] && exit 0   # <-- --dry-run stops here

__profiling_post_run() { declare -F package_post_run && package_post_run; }
trap __profiling_post_run EXIT         # <-- fires even on failure or kill

cd "$PACKAGE_WORKDIR" || exit 77

declare -F package_pre_run && package_pre_run || exit $?

"${cmd[@]}"                            # <-- THE WORKLOAD RUNS HERE
__profiling_status=$?

[ $__profiling_status -eq 0 ] && declare -F profiler_post && profiler_post

exit "$__profiling_status"             # the workload's status, unmodified
```

Four properties this arrangement buys

Property	How
`package_post_run` runs even when the workload dies	It is an EXIT trap, not a line after the call. Verified for a failing workload, a failing pre_run, and a timeout kill
The workload's exit status survives untouched	the resolved command's status is captured immediately and is what the script exits with. A pre_run that returns 42 produces `exit_code: 42` in meta.json
`profiler_post` only runs on success	Guarded on the captured status. Post-processing a profile that was never written produces confusing errors
Every hook is optional	`declare -F` before each call, so `package.sh` may be an empty file

Reading the argv back

The harness writes the final `TARGET_ARGV` to a NUL-separated file that Python reads afterwards. This exists so that `meta.json` records the command that *actually ran* - including any `package_command` rewrite - rather than the one Python guessed before the hooks had their say. The file is deleted once read, and dotfiles at the top of a run directory are excluded from the artifact list regardless.

--dry-run and the honest limit on exactness

"Print the exact command each profiler would execute" is not achievable for an arbitrary bash function without that function's cooperation - the command only exists once the hook has expanded its variables. Rather than approximate it, the contract adds an optional hook:

```
profiler_command() { cmd=(py-spy record --rate "$PYSPY_RATE" ...); }

# --dry-run prints "${cmd[@]}".  The real run executes "${cmd[@]}".
# One builder, one harness, resolved once -- they cannot drift.
```

Both paths share one argv builder, so the printed command provably cannot drift from the executed one. All four shipped profilers implement it. A third-party profiler that does not gets a clearly-labelled fallback naming the wrapper, rather than a plausible-looking guess.

`--dry-run` creates no directories and starts no workload. It writes `target.sh` to a temporary directory, prints the run directory it *would* have used, and exits 0 even when the run would have failed.

SECTION 10

Process control

Tee, timeout, and not leaking processes

Tee-ing output

Two daemon threads copy the child's stdout and stderr to the console and to `stdout.log` / `stderr.log` simultaneously, line by line with an explicit flush. You watch the workload live *and* get the capture. Verified at 50,000 lines with a matching checksum against a direct run, and with multi-byte UTF-8 output.

Timeout

The child is spawned with `start_new_session=True`, giving it its own process group so that a timeout can take the profiler *and* everything it spawned down together. On expiry the group is signalled and the run is recorded as `"status": "timeout"`.

> **That same isolation caused a real bug**
>
> Because the child is in its own session, a signal sent to the harness does **not** reach it. SIGINT, SIGTERM and SIGHUP each left py-spy and the workload running in the container - a cancelled CI job would leak a profiler burning CPU. All three are now handled explicitly and kill the process group on the way out.

And underneath it, a second one

The first fix still leaked. Escalation to SIGKILL was conditioned on the *direct child* still being alive - but GNU `time` sets SIGTERM to `SIG_IGN`, and an ignored disposition survives `exec`. So `time -- sleep 600` left a `sleep` that shrugged off the very signal that killed its parent bash, and SIGKILL never came.

The fix keys escalation on whether the *process group* is empty, probed with signal 0:

```
_signal_group(pgid, SIGTERM)

proc.wait(timeout=term_grace)   # reap the child FIRST - see below

if _group_alive(pgid):          # anything left ignored SIGTERM
    _signal_group(pgid, SIGKILL)
    while time.monotonic() < deadline and _group_alive(pgid):
        time.sleep(0.05)
```

The `proc.wait()` before the probe is not decoration. An un-reaped zombie is *still a member of its process group*, so probing first always reported the group alive and stalled every kill for the full grace period - a 3-second timeout was taking 10 seconds of wall clock. Reaping first brings it to about 3.

The roughly 2 seconds that remain are a deliberate courtesy window: a workload that handles SIGTERM gets a chance to flush partial results before SIGKILL lands.

Verified by process-group id across SIGINT, SIGTERM, SIGHUP and the timeout path, for both a plain exec workload and py-spy's two-process attach mode. Zero survivors in every case.

SECTION 11

Output artifacts

report.py - what a run leaves behind

```
output/
  <package>/<profiler>/<run-id>/
      meta.json      what happened
      target.sh      the exact command, re-sourceable by hand
      stdout.log     tee'd
      stderr.log     tee'd
      <artifacts>    profile.svg, trace.json, out.lprof, time.txt, ...
  <package>/<profiler>/latest -> <run-id>       relative symlink
  summary.json                                  last invocation, overwritten
```

`run-id` is `YYYYmmdd-HHMMSS-<4 hex>`. The `RUN_ID` environment variable overrides it wholesale so CI can use a build number; injected values are sanitised down to `[A-Za-z0-9._-]`, so a value like `../../etc/passwd` cannot escape the output tree.

meta.json

```
{
  "package": "my-tool", "profiler": "py-spy", "run_id": "20260729-141230-a3f1",
  "status": "ok",              # ok | failed | timeout | skipped
  "exit_code": 0, "duration_s": 12.4,
  "started_at": "...", "finished_at": "...",
  "argv": ["..."],             # what actually ran, post package_command
  "kind": "python-module", "workdir": "...",
  "forced": false,             # true if --profiler overrode PACKAGE_PROFILERS
  "flamegraph": "profile.svg", # null unless declared AND the file exists
  "artifacts": ["profile.svg", "stdout.log", "stderr.log"],
  "host": "...", "git_sha": "..."
}
```

`git_sha` is best-effort: if git is missing or this is not a repository, the field is *omitted* rather than the run failing. `flamegraph` is null unless the profiler declares `PROFILER_FLAMEGRAPH=1` *and* the file is actually on disk - declaring one is not the same as producing one.

summary.json

One file per invocation holding the arguments, per-status counts and the array of run records - so a downstream regression gate reads one file instead of globbing a tree.

A bug worth recording here

Re-running a *pinned* `RUN_ID` used to mix artifacts across attempts. Re-running `build-7` with flamegraphs disabled produced a fresh `profile.json` beside the previous attempt's `profile.svg`, and `meta.json` claimed both. The run directory is now cleared first, guarded by a containment check against the output root, and the clearing is announced rather than silent.

SECTION 12

Retention

retention.py - deleting things carefully

`--remove-output` is a terminal action: prune, print what went, exit. `--keep N` keeps N runs **per (package, profiler) pair**, not N in total - so `--keep 3` on a package with four profilers leaves twelve directories. That is what you want when comparing a profiler's history against itself.

This module deletes files, so it is the most defensive code in the tree:

Guard	Behaviour
Output directory sanity	Refuses if PROFILING_OUTPUT_DIR is unset, empty, relative, the filesystem root, or suspiciously shallow
Containment re-check	Every candidate is re-verified to resolve inside the output root immediately before deletion
Unknown directory names	Directories whose names do not match the run-id pattern are **left untouched**, never guessed at
Dry run	`--dry-run` lists exactly what would go and deletes nothing
Symlink repair	`latest` is re-pointed at the newest survivor, or removed when none remains

One consequence is worth stating plainly: a folder dropped into a pair directory *after* the last run counts as the newest run, and is kept until newer runs push it out. Nothing is special-cased -- that is the point.

The bug this module actually had

Survivors were computed as `runs[len(runs) - keep:]`. When `keep` exceeds the run count that index goes negative: `--keep 5` against 3 runs sliced `runs[-2:]` and **deleted the oldest run it had been told to preserve**. It only misfires while count < keep < 2x count, so it would have looked like random data loss. The fix is `runs[-keep:]`, which clamps.

SECTION 13

Exit codes

One number that tells CI what happened

Code	Meaning
`0`	All runs succeeded. **Skips do not affect this.**
`1`	At least one workload failed or timed out
`2`	Usage error: unknown package or profiler, missing --profiler, bad flag, malformed .env
`3`	A required profiler binary is missing
`4`	A package init failed

The harness runs everything and then aggregates - it never stops at the first failure - and reports the **highest** applicable code. If one package's init fails (4) while another package's workload fails (1), the process exits 4.

Why skips are not failures

A skip means a profiler cannot handle a package's kind - asking py-spy to sample a non-Python executable, say. That is a statement about compatibility, not about the code under test. Making it a failure would mean `--package all --profiler all` could never go green, which would make the most useful invocation the least usable one. Skips are recorded in `summary.json` with a reason, so nothing is hidden.

Why a malformed package is exit 2 and not exit 1

Exit 1 means "the code you are profiling has a problem". A package `.env` with a bad `PACKAGE_KIND` is a problem with the *harness configuration*, and a CI gate should treat those differently: one is a regression, the other is a broken pipeline.

SECTION 14

The four profilers

And the one that needed real work

Profiler	Kinds	Flamegraph	Notes
`time`	all three, incl. exec	no	GNU time -v: wall clock, CPU, peak RSS. Cheap enough to always run
`py-spy`	python	**yes**	Sampling. Needs ptrace. See below
`viztracer`	python	no	Deterministic timeline. Large artifacts, heavy overhead
`line-profiler`	python	no	Per-line timings via kernprof. Needs configuring

Flag spellings were verified against the installed tools - py-spy 0.4.2, viztracer 1.1.1, line_profiler 5.0.2 - rather than taken from documentation.

py-spy: the exit status problem

py-spy's exit status describes **py-spy**, not the program it ran, and the two are uncorrelated. Running one command ten times each:

```
child exits 0  ->  py-spy exits  0 1 1 1 1 0 1 1 1 1
child exits 3  ->  py-spy exits  1 0 1 1 1 1 1 0 0 1
```

The 1s on a *succeeding* child come from the flamegraph renderer reporting "No stack counts found" - a function of how long the workload ran, not of whether it worked. Since the run's status *is* the workload's status, trusting py-spy's number would report broken workloads as successful and healthy ones as broken, at random.

So the profiler starts the workload itself and attaches py-spy to the resulting pid. The shell then owns the process and `wait` yields its exact code. The cost is that sampling begins a few milliseconds late.

The obvious alternative - keep launch mode and wrap the workload in a shell that records `$?` - was measured and rejected: the extra fork makes py-spy miss the Python process entirely on **4 of 8** sub-second runs. Losing half the profiles is a far worse trade than a few milliseconds.

Interpreting a non-zero py-spy status

A non-zero status is still used, but only to tell its failure modes apart, and never by trusting the number:

What happened	How it is detected	Outcome
Attached, collected no samples	It still wrote an output file	Warning; the workload's status stands
Could not attach, workload still running	No output file, and the workload was alive when py-spy quit	**Run fails** - ptrace denied or similar
Could not attach, workload already gone	No output file, workload already finished	Warning; the workload's status stands

That last row is the attach-window race, and it is deliberately not a failure. A workload finishing in a few milliseconds occasionally beats the attach; failing on it would make fast packages flaky in CI. Distinguishing the last two rows is only possible while it happens, which is why the profiler waits on py-spy first and then checks the workload's state in `/proc` - a finished-but-unreaped child is still a signalable pid, so `kill -0` cannot answer the question.

viztracer: artifact size

Stock defaults produced **106 MB** of JSON and 17 seconds of overhead from a 0.2-second workload. Capping the circular event buffer brings that to 22 MB and 2.6 seconds. The buffer is circular, so overflowing it costs the earliest events, not the run.

line-profiler: the one that is not zero-config

Without `LINE_PROFILER_TARGETS` (or `@profile` decorators) it produces an empty report. The harness warns - never fails - when a package selects it without configuring it. The warning is driven by a `PROFILER_WARN_IF_UNSET` declaration in the profiler's own `.env`, so the core knows nothing about line-profiler by name and the next tool with the same problem needs no core change.

SECTION 15

Is this too much code?

An honest accounting

2096 lines total, 1655 excluding comments and blanks. That is a fair thing to challenge. Here is where it actually goes.

Module	Lines	What it owns
`cli.py`	`720`	Parser (70), three terminal commands (116), _run_pair (113), cmd_run (71), dry-run printing, helpers, signals
`runner.py`	`583`	One bash harness as a string constant (~55), execute (96), target building (70), dry-run resolution (57), process-group control (60)
`discovery.py`	`236`	Scanning (40), typed accessors (110), selection (35)
`report.py`	`228`	meta.json (60), listings (40), table renderer (25), summary (40)
`retention.py`	`107`	Safety check (25), prune (45), latest symlink (12)
`envfile.py`	`200`	load_layers entry point, sourcing (50), parsing (40), precedence (25)

Roughly a third is features, not core

Retention (107), dry-run resolution (~60), JSON listings and table rendering (~90), summary and meta writing (~90). About 350 lines, and none of it is optional if the tool is to have those features. The actual "run a thing under a profiler" path is closer to 400 lines: discover, layer the environment, build the argv, spawn bash, record the result.

Init used to be on that list at 163 lines. It is now a flag and a `subprocess.run` -- see Section 8.

About 70 lines are bug fixes

Signal handling, the SIGTERM-to-SIGKILL group escalation, and the run-directory reset. Not elegant, but each closes a leak that was demonstrated rather than imagined.

What was redundant, and is now gone

Two things this document previously listed as slop have been removed. `discovery._load_env` and `cli.load_global_env` were the same function differing only in which exception they raised - and both exceptions were caught on one line and produced the same exit code. They are now one `envfile.load_layers`.

And `_print_dry_run` used to call two resolvers backed by two nearly identical bash scripts, spawning two subprocesses for one printed line. They are now one `resolve_dry_run` and one subprocess.

The bigger change is in `cmd_run`: its inner loop body - the eight steps for one (package, profiler) pair - is now `_run_pair`, and `cmd_run` fell from 174 lines to 71. See Section 7.

Where the length is defensible

`cmd_run`'s 174 lines look like the problem and are not, for the reason given in Section 7: the ordering of its steps is the fragile part, and ordering is only visible when the steps are adjacent.

`envfile.py`'s 132 lines look like a lot for "source a file", but the precedence rule and the error attribution are the subtle parts, and both were arrived at by fixing observed problems.

SECTION 16

Testing

111 checks across five suites

Suite	Checks	Covers
`acceptance`	22	The specification's own acceptance list
`bughunt`	21	Quoting torture, hook ordering and failure, package_command, PACKAGE_TIMEOUT=0, malformed .env, unknown profiler, a real directory named 'latest'
`bughunt2`	16	All four environment layers individually, exit-code aggregation, 50k-line tee integrity by checksum, unicode, signal cleanup by process group
`compliance`	33	File tree, stdlib-only imports, every contract variable reaching hooks, cwd, hook ordering, init-once, meta.json fields, run-id format, run ordering
`compliance2`	19	Every config.env override, retention safety refusals, terminal-action behaviour, both py-spy failure classifications

Run twice end to end with no flakes. Two habits proved worth keeping:

Verify the test before believing the failure

Three early "failures" were the tests being wrong, not the code - a `grep -c` counting two lines instead of one, and a glob traversing the `latest` symlink so two runs looked like four. Each was checked before anything was changed.

Beware of measurement artifacts

`pgrep` and `ps | grep` match the measuring shell's own command line. This produced a phantom "py-spy leaks 2 processes" reading. The trustworthy numbers came from probing by process-group id instead.

SECTION 17

Deviations and judgement calls

Where the implementation departs from the specification, and why

1. py-spy uses attach mode, not the spec's literal command

The specification shows `py-spy record -- "${TARGET_ARGV[@]}"`. That form makes the workload's exit status unobservable, as Section 14 shows with measurements. The specification's own instruction to verify flag spellings against installed versions and adjust reads as licence for this, but it is a departure from the literal text and is documented in both the profiler and the README.

2. PATH-like variables are the one exception to layer-4 precedence

Strict precedence would make `PYTHONPATH="$MY_SRC:$PYTHONPATH"` in a package a no-op, contradicting the specification's own example. Section 5 gives the rule; it is called out in the README.

3. --dry-run exactness needs the profiler's cooperation

Achieved through an optional `profiler_dry_run` hook rather than by guessing. All shipped profilers implement it; third-party ones degrade to a labelled fallback instead of printing something that might be wrong.

4. viztracer gains a tracer_entries cap

Beyond the specification's `VIZTRACER_MAX_DEPTH`, because depth alone left 106 MB artifacts. The specification asks for conservative defaults and permits adjusting knobs, so this is within its intent.

Open item

The redundant dry-run harness described in Section 15 is known and unfixed. It is a contained change - delete `_RESOLVE_HARNESS` and `resolve_argv`, have `dry_run_command` return both values in one pass - and would want the 111 checks re-run afterwards.

The through-line

Two ideas carry this design. **Environment layering** means a package can tune a profiler for itself without either knowing about the other. **The target contract** means a profiler can be a prefix wrapper or an interpreter replacement without the core knowing which. Everything else - discovery, retention, init caching, exit codes - is bookkeeping around those two. If you understand Sections 5 and 6, the rest reads itself.